Subject Code	:	CEUE203	Semester	:	3	Academic Year	:	2026-27
Subject Name	:	Object-Oriented Programming						

	Practical 3
Program: 3.1	**1. Distinct points (Point.java + Driver.java with main)** (a) Create class Point with private int x, y and a constructor. (b) Override toString() to return "(x, y)". (c) Override equals(Object) to be true when both x and y match, and hashCode() to agree (e.g. Objects.hash(x, y)). (d) In Driver.main: build a Point[] with some repeated coordinates. (e) Count distinct points: for each point, use equals() to check whether it already appeared earlier; count those that did not. (f) Print "Distinct: N". Expected output: 5 points with 2 repeats → "Distinct: 3".

Code	**Point.java**
	`import java.util.Objects;`
	`public class Point`
	`{`
	`  private int x,y;`
	`  Point(int x,int y)`
	`  {`
	`  this.x=x;`
	`  this.y=y;`
	`  }`
	`  @Override`
	`  public String toString()`
	`  {`
	`  return ("(" + x + "," + y + ")");`
	`  }`
	`  @Override`
	`  public boolean equals(Object obj)`
	`  {`
	`  Point p=(Point) obj;`
	`  return (p.x==x && p.y==y);`
	`  }`
	`  @Override`
	`  public int hashCode()`
	`  {`
	`  return Objects.hash(x,y);`
	`  }`
	`}`

Driver1.java

```
public class Driver1
{
   public static void main(String[] args)
   {
      int i,j,distinct=0;
      boolean found;
      Point[] p=new Point[5];

      p[0] = new Point(2,3);
      p[1] = new Point(5,6);
      p[2] = new Point(1,1);
      p[3] = new Point(2,3);
      p[4] = new Point(5,5);

      for(i=0;i<5;i++)
      {
         found=false;
         for(j=0;j<i;j++)
         {
            if(p[i].equals(p[j]))
            {
               found=true;
               break;
            }
         }
         if(!found)
         {
            distinct++;
         }
      }
      System.out.println("distinct : "+distinct);
   }
}
```

Output

```
C:\materials\3rd sem\OOP java\practicals\practical 3>javac Driver1.java

C:\materials\3rd sem\OOP java\practicals\practical 3>java Driver1
distinct : 4
```

Program: 3.2

Duplicate card check (Card.java + Driver.java) (a) Create class Card with private String rank, suit and a constructor. (b) Override toString() to return rank + " of " + suit (e.g. "Queen of Hearts"). (c) Override equals()/hashCode() by rank + suit. (d) In Driver.main: add cards to a Card[] one at a time; before adding each, use equals() to check it against the earlier cards. (e) Print "Duplicate found: " the first time a repeat appears. Expected output: a second "Ace of Spades" → "Duplicate found: Ace of Spades"

Code

Card.java

```
import java.util.Objects;
public class Card
{
   private String rank, suit;
   Card(String rank,String suit)
   {
      this.rank=rank;
      this.suit=suit;
   }
   @Override
   public String toString()
   {
      return (rank +" of "+suit);
   }
   @Override
   public boolean equals(Object obj)
   {
      Card c=(Card) obj;
      return (rank.equals(c.rank) && suit.equals(c.suit));
   }
   @Override
   public int hashCode() {
      return Objects.hash(rank,suit);
   }
}
```

Driver2.java

```
public class Driver2
{
   public static void main(String[] args)
   {
      int i,j;
      boolean found;
      Card[] c=new Card[5];

      c[0]=new Card("Ten","Spades");
      c[1]=new Card("King","Reds");
      c[2]=new Card("Queen","Diamonds");
      c[3]=new Card("Ace","Clubs");
      c[4]=new Card("Ten","Spades");

      for(i=0;i<5;i++)
      {
         found=false;
         for(j=0;j<i;j++)
         {
            if(c[i].equals(c[j]))
            {
               found=true;
               break;
            }
         }
         if(found)
         {
            System.out.println("Duplicate Found : "+ c[i]);
         }
      }
   }

}
```

Output

```
C:\materials\3rd sem\OOP java\practicals\practical 3>javac Driver2.java

C:\materials\3rd sem\OOP java\practicals\practical 3>java Driver2
Duplicate Found : Ten of Spades
```